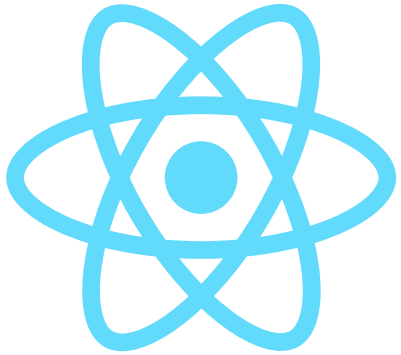


Activity 11 Cart and Bootstrap

Part 1 – Cart 

Part 2 - Bootstrap 

Sarfaraz Ahmed Mohammed, Ph.D.

Product Catalog



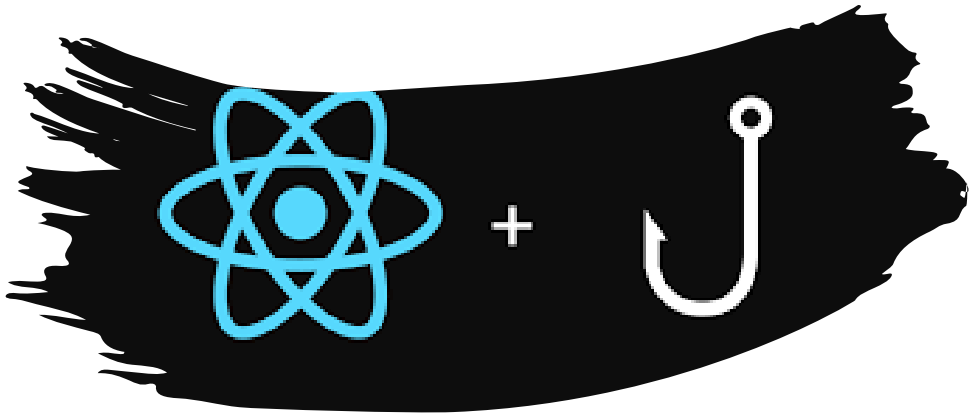
CART

Read JSON files

Reference:
https://dev.to/papasanto/build-a-react-hooks-shopping-cart-with-usestate-and-useeffect-39hk

Let’s start constructing a Cart using Hooks







Long sleeve Jacket
women
150
- +



Jacket with wollen hat
women
65
- +



Compact fashion t-shirt
women
55.99
- +



Blue jins
women
50
- +

We are going to use the Fashion Catalog from Fake Store


```
[
      {
            "_id": 1,
            "title": "Long sleeve Jacket",
            "isNew": true,
            "oldPrice": "200",
            "price": 150,
            "discountedPrice": 135,
            "description": "Lorem ipsum dolor sit amet, consectetur
      adipisicing elit. Nulla non magni facili blanditiis molestias soluta
      eveniet illum accusantium eius mollitia eligendi, ex iste doloribus
      magnam.",
            "category": "women",
            "type": "jacket",
            "stock": 50,
            "brand": "FashionTrend",
            "size": [
              "S",
              "M",
              "L"
            ],
            "image": "https://images.pexels.com/photos/2584269/pexels-
      photo-2584269.jpeg",
            "rating": 4
          },
```


... to continue

Fake Store

https://fakestoreapi.reactbd.com

Construct an `main.jsx` file :

main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import Shop from './Shopping.jsx';



const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <Shop />
  </React.StrictMode>
);
```

Construct a Shopping.jsx Module :

Shopping.jsx

Hooks

```
import React, { useState, useEffect } from "react";

const Shop = () => {


  return (
    <div> Hello </div>
   );
}

export default Shop;
```

Export

Import `Shop` Component from Shopping Module in `main.jsx`:

```jsx
import React from "react";
import ReactDOM from "react-dom/client";
import Shop from "./Shopping.jsx";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <React.StrictMode>
    <Shop />
  </React.StrictMode>
);
```

Continue with `Shopping.jsx` Module :

Add two state variables :

```
import React, { useState, useEffect } from "react";

const Shop = () => {
  const [cart, setCart] = useState([]);
  const [cartTotal, setCartTotal] = useState(0);



  return (
    <div> Hello </div>
   );
}

export default Shop;
```

`cart` is an array which will store the products we are interested in buying.

Import `fashion.json` catalog :

```
import React, { useState, useEffect } from "react";
import item from "./data/fashion.json";

const Shop = () => {
  const [cart, setCart] = useState([]);
  const [cartTotal, setCartTotal] = useState(0);



  return (
    <div> Hello </div>
   );
}

export default Shop;
```

The next functions will be inside the Shop Component :

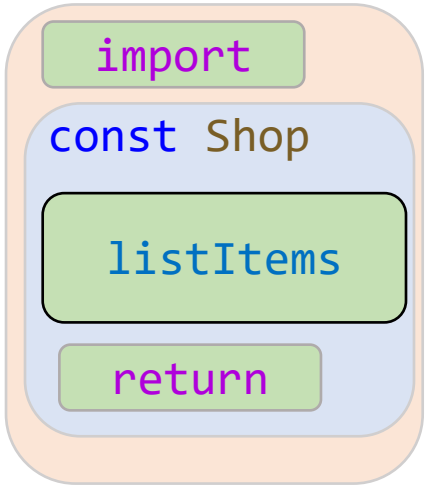
- `const listItems`
- `const addToCar`
- `const removeFromCart`
- `const cartItem`
- `const total`

An the useEffect

- `useEffect(() =>`

Show the Selected Products :

Shopping.jsx




```
  const listItems = items.map((el) => (
    <div key={el.id}>
      <img class="img-fluid" src={el.image} width={150} /> <br/>

      {el.title} <br/>
      {el.category} <br/>
      {el.price} <br/>
    </div>
  ));

  return (
    <div>
      {listItems}
    </div>
  );
};

export default Shop;
```

Temporarily, we can add this : `width={150}`

Add Products to Cart

Add Products to Cart :



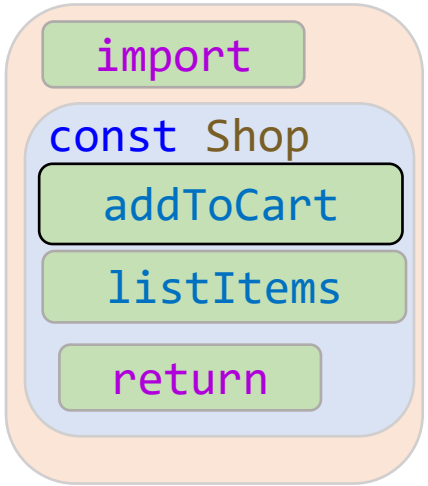


```
const [cart, setCart] = useState([]);
```



Add Products to Cart :

Shopping.js




```
const addToCart = (el) => {
  setCart([...cart, el]);
};
```

setCart modifies cart Hook

Spread operator (...)

products to Cart

Reference: moz://a
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax

Remove Products from Cart

Remove Products from Cart :





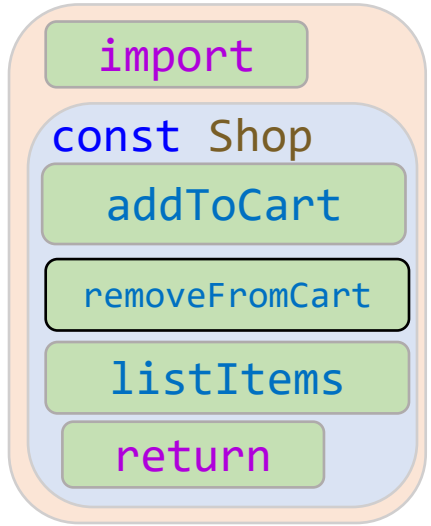
In reality, we do not return the product to the catalog, we just remove it from Cart.



```
const [cart, setCart] = useState([]);
```

Remove Products from Cart :

Shop.js




```
const removeFromCart = (el) => {
    let hardCopy = [...cart];
   hardCopy = hardCopy.filter((cartItem) => cartItem.id !== el.id);
    setCart(hardCopy);
};
```

Note, that we make a copy of the Cart state before replacing it.

```
// fixing removing only one item from cart

const removeFromCart = (el) => {
  let itemFound = false;
  const updatedCart = cart.filter((cartItem) => {
     if (cartItem.id === el.id && !itemFound) {
         itemFound = true;
         return false;
     }
     return true;
    });
    if (itemFound) {
      setCart(updatedCart);
    }
 }
```

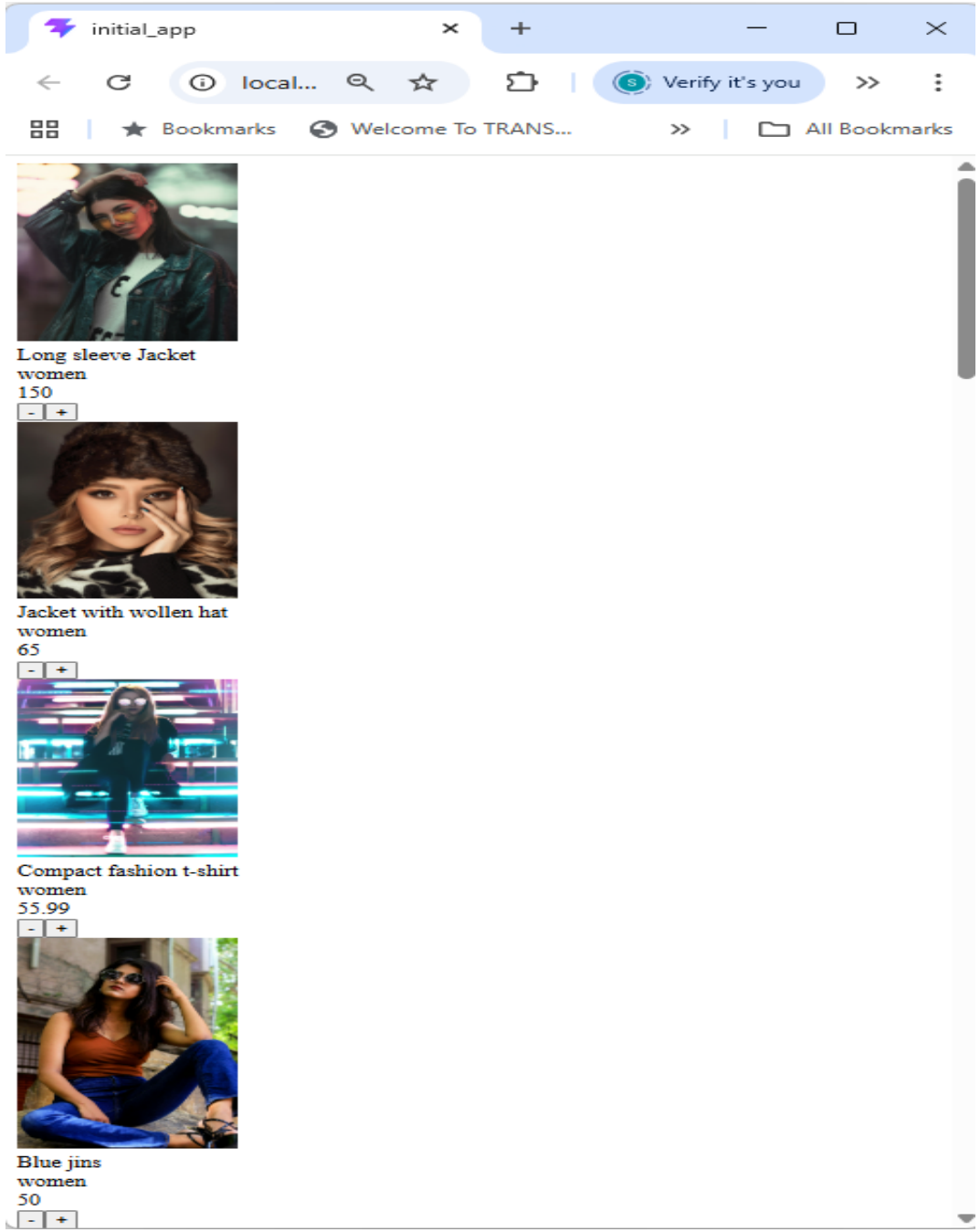
Add one <button to add and one <button to remove items (products) from Cart :

```
const listItems = items.map((el) => (
  <div key={el.id}>
    <img class="img-fluid" src={el.image} width={100} />
    {el.title}
    {el.category}
    {el.price}
    <button type="button" onClick={() => removeFromCart(el)}>-</button>{" "}
    <button type="button" variant="light" onClick={() => addToCart(el)}> + </button>
  </div>
));
```

```
const addToCart = (el) => {
  setCart([...cart, el]);
};
```

Observe, that every Click in the “+” symbol `addToCart` is called.

```
<button type="button" variant="light" onClick={() => addToCart(el)}> + </button>
```



Sarfaraz Ahmed Mohammed, Ph.D.

Show the content of the Cart

Show the content of the Cart :

Shopping.js

- import
- const Shop
  - addToCart
  - removeFromCart
  - cartItems
  - listItems
  - return

```
const cartItem = cart.map((el) => (
  <div key={el.id}>
  <img class="img-fluid" src={el.image} width={150} />
    {el.title}
    ${el.price}
  </div>
));
```

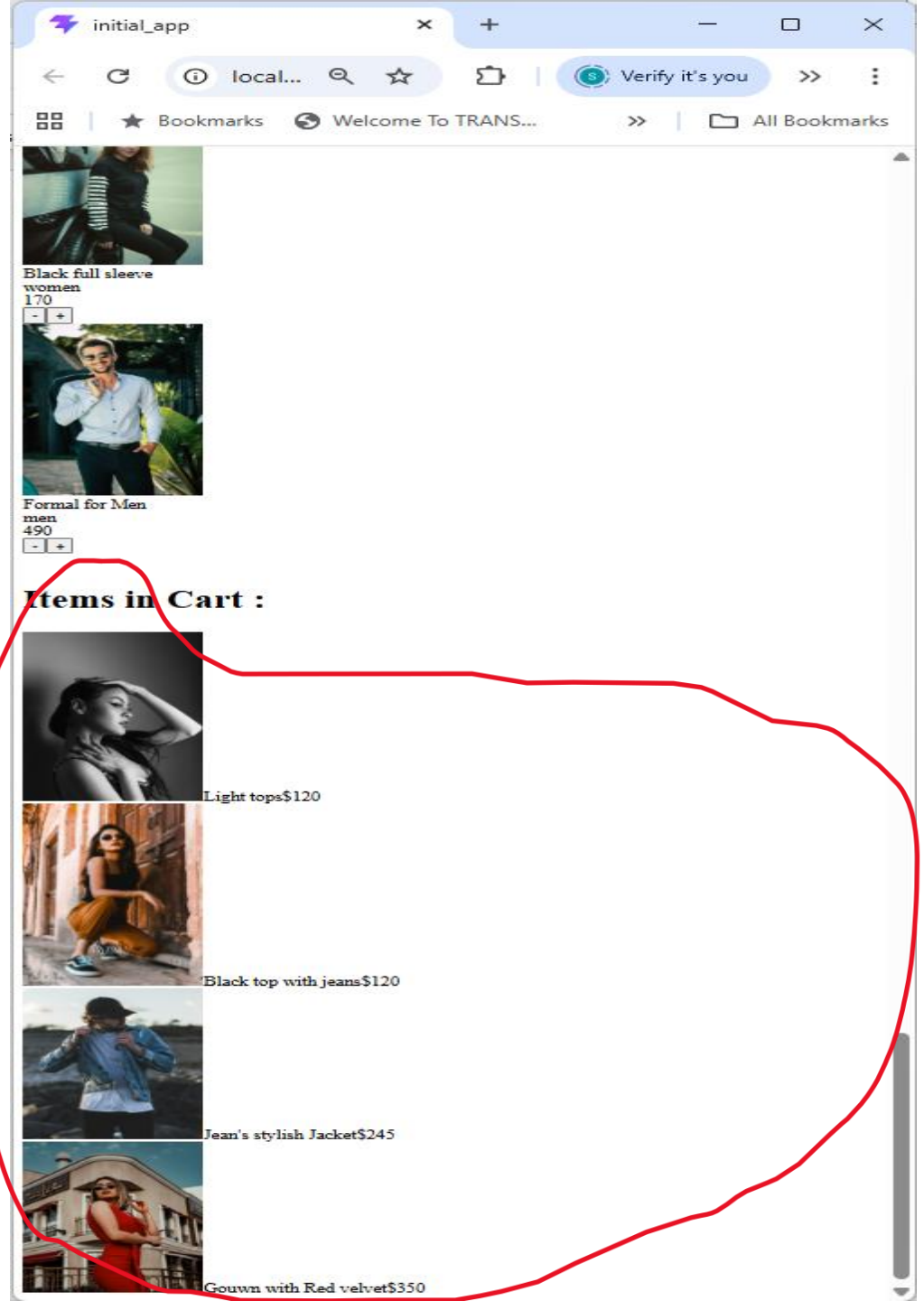
Display the information that you prefer ...

This function won't be used in the final Shopping Cart.



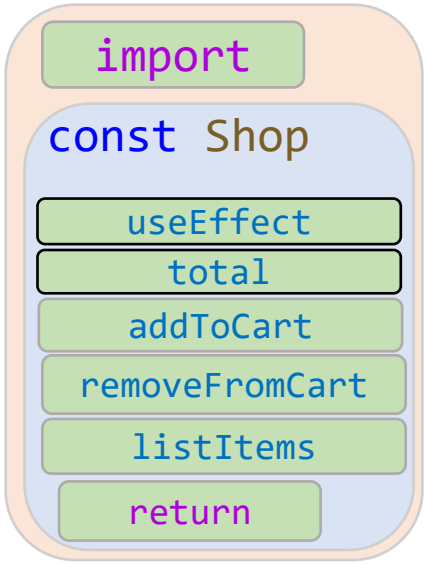
```
return (
  <div>
    <div>{listItems}</div>
    <h1>Items in Cart :</h1>
    <div>{cartItem}</div>
  </div>
);
```

We can show the items in the Cart :



Compute the Order total to pay

Shopping.js




```
useEffect(() => {
  total();
}, [cart]);
```

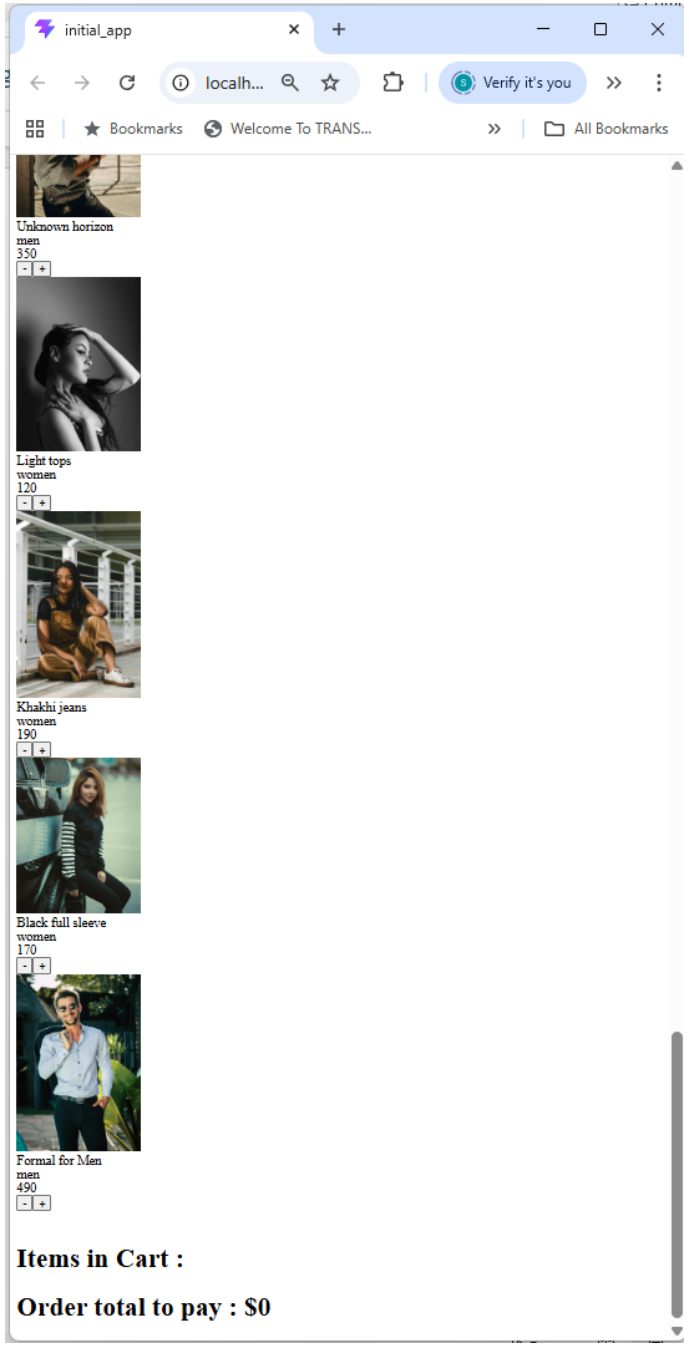
Observe that every time the state variable `cart` changes, `useEffect` is called.

```
const total = () => {
  let totalVal = 0;
  for (let i = 0; i < cart.length; i++) {
    totalVal += cart[i].price;
  }
  setCartTotal(totalVal);
};
```

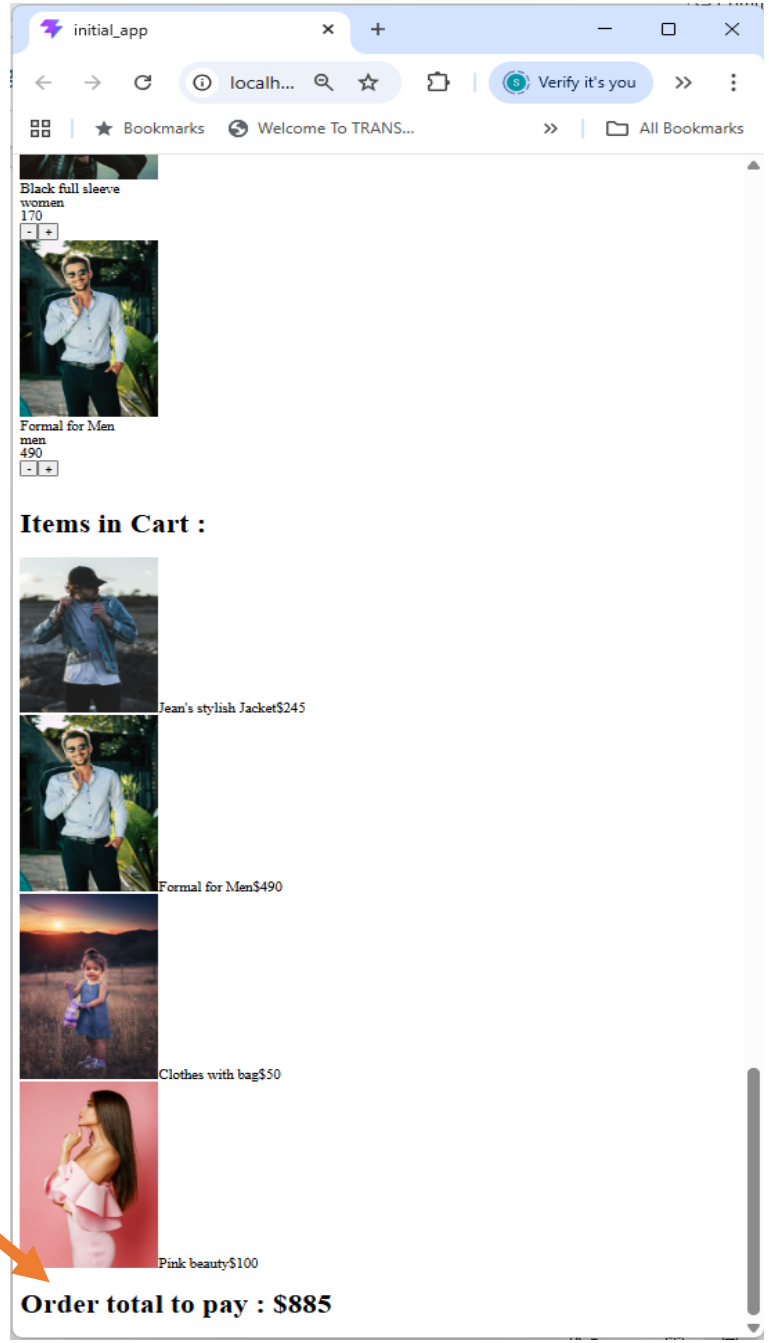
There, `total` is executed where the prices are added and finally `carTotal` is updated.

```
const [cartTotal, setCartTotal] = useState(0);
```

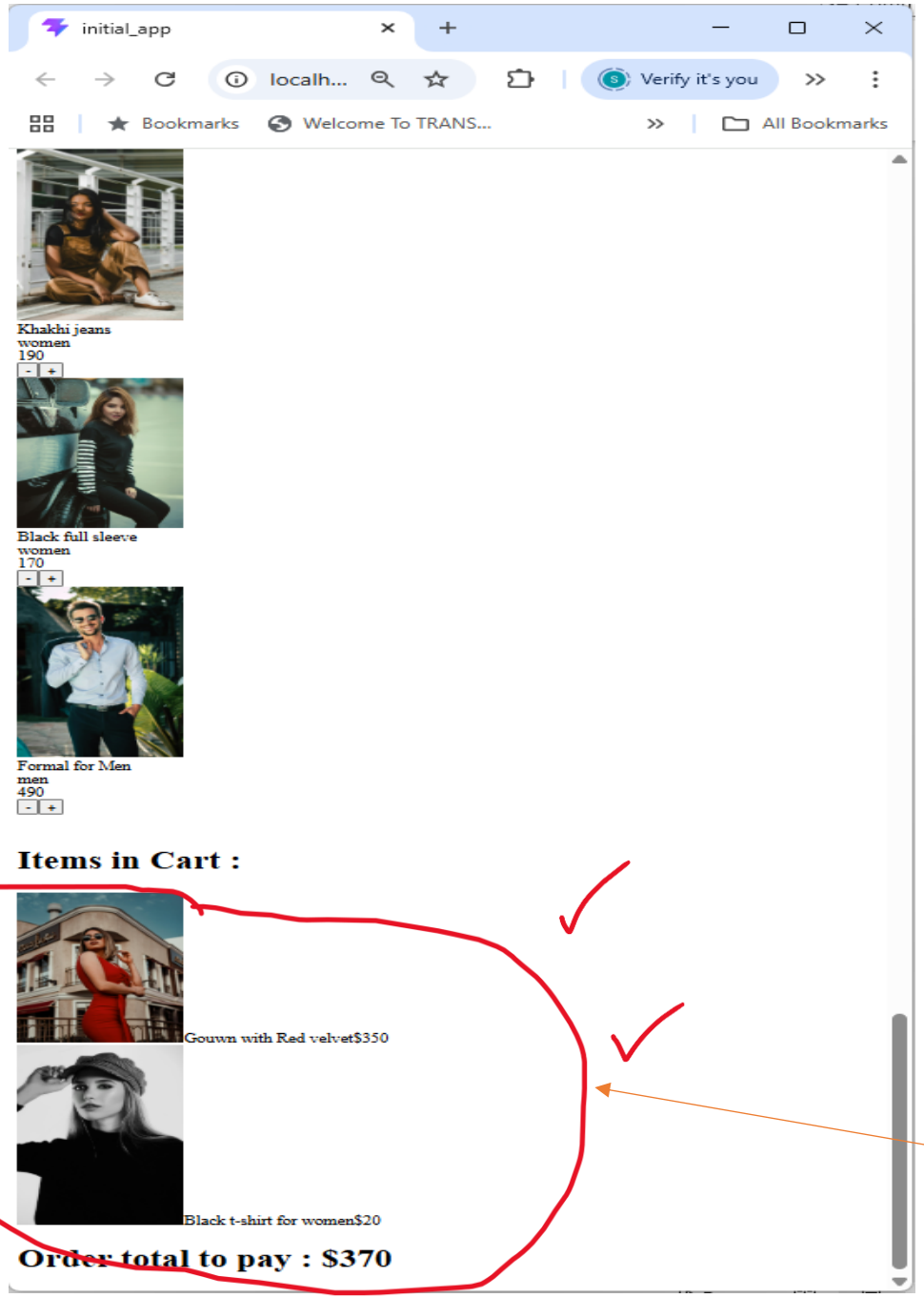
```
return (
  <div>
    <div>{listItems}</div>
    <h1>Items in Cart :</h1>
    <div>{cartItem}</div>
    <h1>Order total to pay :${cartTotal}</h1>
  </div>
);
```



It shows the Order Total :



Test the Shopping Cart



Add + and Remove - some products and observe the items in the Cart …

This section won't be part of the Shopping Cart at the end.